

Secure and Safe Use of OpenClaw with Capability Tradeoffs

Executive summary

Data collection date: March 14, 2026 (America/Los_Angeles). This report reflects project documentation, the Trust threat model, and the public advisory stream available on that date. ¹

OpenClaw is best understood as a **high-privilege automation control plane** (gateway + tools + skills + channels), built primarily for a **single trusted operator boundary** (“personal assistant” model), not for hostile multi-tenant sharing. ² A secure deployment therefore hinges less on “prompt safety” and more on **hard boundaries**: who can reach the gateway, who can message the agent, which tools are enabled, where execution runs (host vs sandbox), and which third-party code (skills/plugins) is allowed. ³

OpenClaw’s recent vulnerability history reinforces three security lessons that shape best practices:

- **Localhost is not a safety boundary when the browser is involved.** Recent advisories include “1-click token exfiltration” via a crafted URL and browser-to-loopback paths affecting WebSocket and browser relay endpoints. ⁴
- **Reverse proxy / trusted-proxy mode raises the stakes.** A recent issue enabled untrusted web origins to inherit proxy-authenticated identity (patched), and community reports show operational pitfalls (e.g., breaking CLI/sub-agent auth in trusted-proxy mode). ⁵
- **Supply chain is a first-class risk.** OpenClaw documentation explicitly treats third-party skills as untrusted code, and OpenClaw added automated marketplace scanning with a clear “not a silver bullet” caveat. ⁶

This report frames secure usage as a **deliberate tradeoff between security controls and agent “agency”** (automation scope, autonomy, and feature breadth). It provides two reference postures:

- **Conservative posture (maximum safety):** loopback-only gateway, strict DM pairing/allowlists, default-deny tools, sandboxed execution with no network, no host browser control, no third-party skills unless reviewed/pinned. This sharply reduces “hands-free” automation and web/browsing capability, but minimizes the blast radius of prompt injection and supply-chain compromise. ³
- **Permissive posture (maximum agency with guardrails):** allows web/browsing, controlled exec, and selected skills—but **only** with strong compensating controls: sandboxing, egress restrictions, per-channel/per-sender tool restrictions, rigorous skill onboarding, and monitoring/rapid patching. ⁷

Where controls conflict with maximizing agent capability, this report gives **alternatives** (e.g., VPN/SSH tunnel instead of public exposure; sandbox browser instead of host browser relay; per-sender tool policies instead of global disables) and **risk acceptance criteria** (e.g., “only acceptable on a dedicated machine with no long-lived secrets”). ⁸

Project overview and architecture

OpenClaw's docs describe a gateway-driven system with strict configuration validation (unknown keys/invalid values prevent boot), tool gating, channels (DM + group policies), and optional sandboxing that moves tool execution into containerized environments while keeping the gateway control plane on the host.

⁹ Key architectural security boundaries:

- **Gateway:** control plane for auth, sessions, routing, tool policy, and API endpoints. ¹⁰
- **Tools:** capability surface (filesystem, runtime, browser, messaging, nodes, cron). Tools can be restricted globally and per-agent using profiles plus allow/deny lists ("deny wins"), and further restricted by provider/model. ¹¹
- **Channels:** ingress paths (DM/group) with explicit DM policies (pairing/allowlist/open/disabled) and group policies/mention gating. ¹²
- **Sandboxing:** opt-in execution isolation; defaults include **no network** for sandbox containers unless configured otherwise, and specialized sandbox browser controls (separate network, CDP ingress restrictions, optional host-browser control). ¹³
- **Skills / plugins:** third-party extendability; docs explicitly warn to treat third-party skills as untrusted and note secret injection behaviors. ¹⁴

High-level trust boundary statement (critical for safe deployments): OpenClaw's official security guidance emphasizes a **single trusted operator boundary per gateway**; if multiple mutually untrusted people can message a tool-enabled agent, they effectively share the same delegated authority, and isolation requires separate gateways (ideally separate OS users/hosts). ²

Threat model recap and capability-sensitive risk analysis

Trust model recap

OpenClaw's Trust threat model is explicitly based on the MITRE ¹⁵ ATLAS framework and enumerates dozens of threats with multiple critical/high-risk items, emphasizing prompt injection, tool misuse, identity/auth risks, and supply-chain compromise via skills/marketplaces. ¹⁶

OpenClaw's own security documentation stresses that prompt injection is not "solved" by system prompts; hard enforcement comes from channel controls, tool policy, exec controls, sandboxing, and careful handling of untrusted content (links/attachments/instructions). ¹⁷

What "maximizing agency" changes in threat exposure

"Agency" (more tools, more network access, more autonomy) increases exposure in predictable ways:

- Enabling browsing/web tools increases the probability of the agent consuming attacker-controlled content and following malicious instructions (indirect prompt injection), and increases the value of SSRF protections and egress control. ¹⁸
- Enabling exec/runtime tools increases impact from any authorization mistake or policy bypass. OpenClaw's advisory stream includes multiple recent authorization/policy issues (e.g., scope binding and origin validation). ¹⁹

- Enabling third-party skills introduces a supply-chain boundary; OpenClaw explicitly treats them as untrusted and added marketplace scanning while warning it cannot catch everything (especially natural-language coercion/prompt attacks). ⁶
- Exposing the gateway beyond loopback (or via reverse proxy/trusted-proxy) expands attacker positions from “local” to “network/web.” Recent issues show browser-originated WebSocket paths can matter even when the gateway is loopback-bound. ²⁰

Representative attack chains that motivate best practices

```

flowchart LR
    A[Untrusted content enters via channel/webhook/web] --> B[Prompt injection / instruction hijack]
    B --> C{Are high-risk tools enabled?}
    C -- No --> D[Limited impact: text-only / low-scope actions]
    C -- Yes --> E{Is execution sandboxed?}
    E -- Yes --> F[Containment: workspace-scoped + limited egress]
    E -- No --> G[Host-level impact: file/exec/credentials]
    G --> H[Credential theft / config change / persistence]

```

This model matches OpenClaw’s stated “hard controls matter” stance (channels + tools + sandboxing), and aligns with the emphasis in the Trust threat model on tool misuse and supply-chain compromise as core risks. ²¹

Recent advisories illustrate concrete breakpoints in those chains:

- Browser-to-gateway token exfiltration via Control UI (`gatewayUrl`) enabling privileged gateway control. ²²
- Browser relay `/cdp` missing auth allowing loopback WebSocket access to sensitive browser data. ²³
- Cron webhook SSRF enabling access to private/metadata/internal endpoints when SSRF checks were bypassed. ²⁴
- Trusted-proxy origin validation bypass allowing untrusted origins to inherit proxy-authenticated identity (patched). ²⁵
- Shared-auth WebSocket scope binding flaw allowing elevated scopes to be self-declared (patched). ²⁶

Best-practice controls with explicit tradeoffs

This section is organized by control domain. For each control, you get: security benefit, operational cost, and impact on capabilities, plus alternatives when agency is required.

Control matrix

Terms used: - **Security benefit:** what risk it reduces. - **Operational cost:** setup complexity, maintenance, friction, and runtime overhead. - **Capability impact:** what OpenClaw can’t do (or does slower/less autonomously).

Control domain	Control option	Security benefit	Operational cost	Capability impact	When to choose	Alternatives: risk acceptance criteria
Network	Loopback-only gateway (<code>bind: loopback</code>)	Minimizes network attack surface; reduces exposure to proxy/origin mistakes	Remote access requires tunneling/VPN	Less convenient remote UI/control	Default for personal assistant setups	If remote access needed: keep loopback and use VPN/SSH tunnel; avoid public exposure. ²⁷
Network	Reverse proxy + <code>trusted-proxy</code> auth	Enables SSO/identity-aware proxy policies	High complexity; misconfig risk; requires strict proxy boundary	Can break some local CLI/sub-agent paths in practice	Only when you need enterprise identity and can enforce proxy trust	Avoid unless required; if used, lock trusted proxies and validate with audit; be aware of known operational pitfalls. ²⁸
Auth / ingress	DM policy = pairing/allowlist	Prevents unknown senders from steering tool authority	Onboarding friction; admin overhead for shared inboxes	Slower initial UX for new senders	Default safe approach	If “open” DMs required: pair with per-sender tool restriction and sandboxing; accept that prompt injection risks. ²⁹

Control domain	Control option	Security benefit	Operational cost	Capability impact	When to choose	Alternatives: risk acceptance criteria
Channels	Group allowlist + mentioning default	Reduces opportunistic prompting in group rooms	Requires configuration + user behavior	Less ambient automation	Default for any group usage	If must be always-on in a group: restrict tool heavily; isolate sessions per channel peer. ³⁰
Tools	<code>tools.profile=minimal</code> + selective allow	Prevents accidental tool exposure to model/provider	More config work per use case	Reduced autonomy; more manual steps	High-security deployments, shared inboxes	For high agency: use <code>coding</code> / custom profiles but deny the riskiest tool on untrusted inputs. ¹¹
Execution	Sandbox enabled; “no network” default	Contains file/exec impact; limits exfil and SSRF	Container runtime dependency; possible overhead	Web/browsing may fail without egress	Default for untrusted inputs or shared inboxes	If web needed: enable sandbox egress but restrict destination or use proxy with allowlist. ¹
Browser	Sandbox browser target	Reduces risk of host browser compromise paths	Setup overhead	Some host-only browser workflows unavailable	Default if browser tool is enabled	Avoid host browser relay unless necessary; used, treat as high risk and keep patched. ³

Control domain	Control option	Security benefit	Operational cost	Capability impact	When to choose	Alternatives: risk acceptance criteria
Supply chain	No third-party skills	Eliminates marketplace supply-chain class	Reduced feature breadth	Less capability/automation	High-assurance environments	If skills needed: internal review + pin versions + monitor; do not trust "scan clean" alone. ⁶
API endpoints	Disable OpenResponses/ OpenAI-compatible HTTP APIs by default	Removes a high-value auth surface	Limits programmatic use	Less automation via HTTP	Default unless you explicitly need it	If enabled: keep on loopback/ private ingress; treat token as operator credential. ³²

Network and exposure controls

Keep the gateway loopback-only unless you have a controlled private network boundary. This aligns with OpenClaw's security posture ("personal assistant trust model") and reduces exposure to WebSocket-origin and proxy-related vulnerabilities. ³³

- Benefit: shrinks external attack surface. - Cost: remote UI/control requires a tunnel/VPN. - Capability impact: reduced convenience; not reduced *core* capability.

If you must use a reverse proxy, treat `trusted-proxy` mode as an advanced configuration with both security and operational hazards. OpenClaw provides a dedicated trusted-proxy auth guide and explicitly recommends testing your proxy independently and running `openclaw security audit`. ³⁴

- Benefit: enterprise identity and per-user authentication at the proxy layer. - Cost: higher misconfig risk; recent critical issues were specifically tied to trusted-proxy + origin validation. ²⁵

- Capability impact: community reports show trusted-proxy can break local CLI/sub-agent features without additional accommodation. ³⁵

- Alternative: keep token auth + VPN/tailnet; accept lack of per-user identity inside gateway but retain simpler operational behavior.

Risk acceptance criteria for public exposure: If you decide to expose gateway endpoints publicly (strongly discouraged by OpenClaw's own warnings about operator credential scope), acceptable only when (a) gateway is hardened and continuously patched, (b) external auth is enforced (IAP/SSO), (c) tool policy is minimized, (d) execution is sandboxed, and (e) there are no long-lived high-value secrets on that runtime. ³⁶

Authentication, ingress gating, and session isolation

Use pairing/allowlists for DMs; avoid “open” DM mode unless you can tolerate prompt injection attempts. OpenClaw’s configuration reference defines DM policies and notes that “open” requires wildcard allowFrom, while pairing is the default safe mode. ³⁷

- Benefit: reduces attacker ability to steer tool authority. - Cost: onboarding friction and administrative approvals. - Capability impact: slows “instant access” for new users.

Use group allowlists and keep mention gating enabled by default. OpenClaw group docs describe a fail-closed allowlist posture and mention gating as default behavior. ³⁸

- Benefit: reduces ambient triggering in groups and limits opportunistic attack surface. - Cost: configuration and user training (“mention the bot”). - Capability impact: less autonomous background activity.

For shared inboxes (cooperative, not adversarial), split sessions by DM peer / channel peer. OpenClaw’s security audit explicitly recommends secure DM scoping (`session.dmScope="per-channel-peer"` or similar) and reiterates that adversarial multi-user sharing is not a supported security boundary. ³⁹

- Benefit: reduces cross-user privacy leakage and limits session contamination. - Cost: more sessions to manage; more state. - Capability impact: less “single shared memory” convenience.

Tooling controls and least privilege

Start with a restrictive tool profile and explicitly add capability—don’t start from “full.” OpenClaw documents tool profiles (`minimal`, `coding`, `messaging`, `full`) and global allow/deny controls where deny wins. ¹¹

Practical best practice is to treat tool enabling as “granting delegated authority,” especially for runtime/filesystem/browser tools.

- Benefit: prevents accidental exposure of high-impact tools.
- Cost: more upfront configuration and iterative tuning.
- Capability impact: less agent autonomy until you add the needed tools.

Restrict tools based on ingress trust. A reliable pattern is:

- Trusted operator DM → broader tools (still preferably sandboxed).

- Any group or shared inbox → reduced tool set; ideally no exec and no broad filesystem.

OpenClaw supports per-channel and per-sender tool restrictions in channel configurations (shown in channel docs as examples of denying high-risk tool groups). ⁴⁰

Assume “policy bypass” is a recurring class and keep patched. OpenClaw’s advisory stream shows repeated authorization and control-plane hardening fixes (scope binding, origin validation, command/policy issues). ⁴¹

- Benefit: reduces reliance on brittle parsing/allowlist logic. - Cost: requires rapid upgrades (days, not quarters). - Capability impact: none directly, but patching cadence is operational work.

Sandboxing and execution isolation

Enable sandboxing for any scenario involving untrusted content or shared ingress. OpenClaw's sandboxing docs describe tool execution in containers, note default "no network" behavior for sandbox containers, and provide guardrails for sandbox browser usage (dedicated networks, CDP restrictions, allowlists for custom control targets). ¹³

- Benefit: reduces host compromise blast radius and provides workspace scoping. - Cost: requires container runtime and image management; may add latency and complexity. - Capability impact: web-enabled workflows may fail under no-egress defaults unless you configure controlled egress.

Avoid `scope: "shared"` **unless you explicitly accept cross-session contamination.** OpenClaw's Docker/sandbox documentation warns that shared scope disables cross-session isolation (all sessions share a container/workspace). ⁴²

- Benefit of shared scope: lower overhead, faster repeated tasks. - Security cost: larger blast radius and higher data leakage risk. - Capability benefit: more seamless multi-session "memory" and lower latency.

Be cautious with host bind mounts into sandboxes. OpenClaw's FAQ notes that bind mounts bypass "sandbox filesystem walls" and recommends read-only binds for sensitive paths. ⁴³

- Benefit: enables powerful real-world workflows (codebase access). - Security cost: undermines containment if writable. - Capability impact: none; it increases capability, but at the cost of isolation.

Browser capability controls

OpenClaw's browser tool docs describe two broad control modes: sandbox browser (typical in sandboxed sessions) and host browser control (requires explicit configuration). ⁴⁴

Preferred pattern: sandbox browser for browsing automation.

- Benefit: reduces risk of host browser compromise; aligns with containment. - Cost: setup and possible feature gaps compared to a fully interactive host browser. - Capability impact: some host-only workflows (extensions, logged-in sessions) may be unavailable.

High-risk pattern: host browser relay / CDP. A recent CVE showed that a loopback CDP WebSocket endpoint could be reachable without auth in certain versions, enabling cookie theft and cross-tab script execution. ²³

Risk acceptance criteria for enabling host browser control: - Only on a dedicated machine/profile with minimal sensitive sessions. - Only on fully patched versions. - Only when the workflow truly requires host context (e.g., existing logged-in session that cannot be replicated safely in a sandbox). ⁴⁵

Skills, plugins, and supply chain governance

Treat third-party skills as untrusted code and review before enabling. OpenClaw's skills docs say this directly, and note that some skill secrets are injected into the host process for that agent turn (not the sandbox), increasing the importance of minimizing secrets and guarding logs/prompts. ⁴⁶

- Benefit: reduces supply-chain compromise probability and limits credential leakage. - Cost: review and approval workflow. - Capability impact: slower onboarding of new automations.

Use marketplace scanning as a signal, not a guarantee. OpenClaw announced VirusTotal ⁴⁷ scanning (including Code Insight + daily re-scans) but explicitly states it is not a silver bullet because prompt injection and natural-language coercion may not be signature-detectable. ⁴⁸

Recommended practice: - pin skill versions and maintain a local allowlist registry; - block skills that download/execute external code unless explicitly approved; - periodically re-review high-privilege skills after updates. ⁴⁸

Lock down plugin loading paths. OpenClaw's plugin docs support allow/deny lists and loading extra plugin paths; treat any extra plugin path as "code execution from that directory." ⁴⁹

CI/CD and automation safety

OpenClaw's CLI includes security and secrets auditing commands, and its security audit explicitly flags risky shared-ingress configurations, webhook sessionKey override risks, and dangerous tool exposure patterns. ⁵⁰

When integrating OpenClaw into CI/CD or automated environments:

- Use ephemeral runners/hosts; never reuse a long-lived gateway token across environments (treat as operator credential). ³²
- Disable or tightly restrict high-risk tools in CI contexts; use sandboxing with no network unless the job requires outbound access. ⁵¹
- Run `openclaw secrets audit` and keep secrets out of prompts/logs, consistent with the skills secret-injection warning. ⁵²

Logging, monitoring, and incident response controls

Use the built-in security audit as a continuous control, not a one-time setup check. OpenClaw provides `openclaw security audit`, `--deep` live probing, and `--fix` to tighten defaults and filesystem permissions. ⁵³

- Benefit: catches "foot-guns" during drift (open DM policies, risky hooks, sandbox misalignment, dangerous tool exposures). - Cost: small; integrate into cron/CI and change control. - Capability impact: none; may enforce safer defaults that reduce some convenience.

Monitor egress for SSRF/exfil patterns when automation and webhooks are enabled. The cron webhook SSRF advisory shows that webhook delivery paths can become SSRF vectors (fixed in later versions), reinforcing the need for egress monitoring and URL allowlists. ²⁴

Incident response baseline (operator-credential model): if gateway token/password is suspected compromised, treat as full operator compromise; OpenClaw's HTTP API docs explicitly say valid tokens/passwords should be treated as operator credentials. ³²

Minimum playbook: - isolate gateway network access; - rotate gateway credentials and connected provider keys; - review config changes and installed skills/plugins; - run `openclaw security audit --deep` and `openclaw secrets audit`. ⁵²

Secure configuration checklist and reference configurations

Prioritized secure configuration checklist

Highest impact / lowest effort - Keep gateway on loopback; avoid public exposure. ⁵⁴

- Use DM pairing/allowlists; avoid “open” DM policies unless you restrict tools and sandbox. ⁵⁵
- Set conservative tool profile (`minimal` or a custom allowlist) and deny high-risk tools by default (“deny wins”). ¹¹
- Enable sandboxing for non-main sessions; keep no-network default unless required. ¹³
- Avoid third-party skills unless reviewed; pin versions; keep secrets out of prompts/logs. ⁴⁶
- Run `openclaw security audit` regularly; run `--fix` after changes to re-tighten permissions and defaults. ⁵³

High impact / moderate effort - Split trust boundaries: separate gateways (and ideally separate OS users/hosts) for untrusted users or mixed-trust workloads. ²

- If you must enable browser automation, prefer sandbox browser and restrict host control; keep patched due to prior browser relay auth issues. ⁵⁶
- Keep OpenResponses/OpenAI-compatible HTTP endpoints disabled unless required; if enabled, keep on private ingress and treat bearer as operator. ³²

Situational / advanced - Use trusted-proxy only when needed, and harden proxy trust boundaries; validate with audit and targeted tests. ⁵⁷

- For webhook/hook ingestion, avoid unsafe “external content bypass” flags except for scoped debugging (they disable safety wrapping). ⁵⁸

Reference configuration: conservative posture (secure-by-default)

```
// ~/.openclaw/openclaw.json (JSON5)
{
  // Network posture
  gateway: {
    bind: "loopback",
    // keep default port unless you have a reason to change
    auth: { mode: "token" },
    // keep HTTP endpoints disabled unless explicitly needed
    http: { endpoints: { responses: { enabled: false } } },
  },

  // Channels: DM pairing/allowlist; groups fail-closed + mentions
  channels: {
    defaults: {
      groupPolicy: "allowlist",
    },

    // Example pattern: explicitly configured channels only.
    // For each configured channel provider:
```

```

    // - dmPolicy: "pairing" (default) or "allowlist"
    // - groups: allowlist only; requireMention true
  },

  // Minimal tool posture
  tools: {
    profile: "minimal",
    // deny wins even if agent tries to enable tools.
    deny: [
      "group:runtime",
      "group:fs",
      "browser",
      "canvas",
      "nodes",
      "cron",
    ],
    elevated: { enabled: false },
  },

  // Execution: sandboxed, workspace-scoped, no egress by default
  agents: {
    defaults: {
      sandbox: {
        mode: "non-main",
        scope: "session",
        workspaceAccess: "rw",
        docker: {
          network: "none",
        },
      },
      // For highest isolation, avoid host binds unless read-only and
      // necessary.
    },
  },

  // Skills: keep disabled or explicitly allow only reviewed/pinned skills
  skills: {
    enabled: false,
  },

  // Plugins: allowlist only (or disabled)
  plugins: {
    enabled: true,
    allow: [],
  },
}

```

This posture matches OpenClaw's guidance on the personal assistant model, tool allow/deny semantics, opt-in sandboxing, and the recommendation to treat third-party skills as untrusted. ³

Reference configuration: permissive posture (high agency with compensating controls)

```
{
  gateway: {
    bind: "loopback",
    auth: { mode: "token" },

    // Enable HTTP APIs only on private ingress and only if needed.
    http: { endpoints: { responses: { enabled: true } } },
  },

  tools: {
    profile: "coding", // or custom profile tuned to your workflows
    deny: [
      // Keep the most dangerous control-plane tools restricted unless needed
      "gateway",
      // Consider denying nodes/cron unless required
    ],

    // Provider/model-specific restriction example
    byProvider: {
      "some-risky-provider": { profile: "minimal" },
    },

    elevated: {
      enabled: false, // keep host-elevated off unless explicitly required
    },
  },

  agents: {
    defaults: {
      sandbox: {
        mode: "non-main",
        scope: "agent", // lower overhead than per-session, higher blast radius
        workspaceAccess: "rw",
        docker: {
          // If you need web access, do NOT use "none".
          // Prefer a dedicated network + firewall/egress restrictions outside
          OpenClaw.
          network: "openclaw-sandbox-egress",
        },

        browser: {
```

```

        // Prefer sandbox browser; avoid host control unless required.
        autoStart: true,
        network: "openclaw-sandbox-browser",
        // If you must allow host browser control, do it consciously:
        allowHostControl: false,
    },
},
},
},

// Skills on, but only allow reviewed/pinned skills; treat marketplace scans
as signal.
skills: {
    enabled: true,
    allow: [
        // "org/reviewed-skill@1.2.3"
    ],
},
}

```

Key design idea: this posture grants meaningful agency (runtime + filesystem + browser) but relies on sandboxing and policy gating as hard controls, consistent with OpenClaw's security guidance. ⁵⁹

Ambiguity note: OpenClaw's docs provide the primitives (profiles, allow/deny, sandbox network controls), but "best" egress restriction implementation depends on your host firewall/proxy architecture and is not fully prescriptive in the referenced documentation. Validate in your environment and treat as a required engineering step, not a toggle. ¹³

Automated hardening scripts and IaC snippets

These are pragmatic baselines. Adapt paths and service names to your operating system and your OpenClaw installation method. The intent is to codify: loopback binding, strong credential generation, least-privilege runtime user, and continuous auditing consistent with OpenClaw's configuration/validation model.

⁶⁰

Bash hardening (host baseline + continuous audit)

```

#!/usr/bin/env bash
set -euo pipefail

# Adjust to your environment
OC_USER="openclaw"
OC_HOME="/var/lib/openclaw"
OC_CFG_DIR="/etc/openclaw"
OC_PORT="18789"

```

```

# 1) Dedicated low-privilege user
if ! id -u "$OC_USER" >/dev/null 2>&1; then
    useradd --system --create-home --home-dir "$OC_HOME" --shell /usr/sbin/
nologin "$OC_USER"
fi

# 2) Directories with tight permissions
install -d -m 0750 -o root -g "$OC_USER" "$OC_CFG_DIR"
install -d -m 0700 -o "$OC_USER" -g "$OC_USER" "$OC_HOME"

# 3) Generate gateway token (operator credential)
TOKEN="$(openssl rand -hex 32)"
cat > "$OC_CFG_DIR/secrets.env" <<EOF
OPENCLAW_GATEWAY_TOKEN=$TOKEN
EOF
chmod 0640 "$OC_CFG_DIR/secrets.env"
chown root:"$OC_USER" "$OC_CFG_DIR/secrets.env"

# 4) Conservative config (loopback + restricted tools + sandbox)
cat > "$OC_CFG_DIR/openclaw.json" <<'JSON'
{
  "gateway": {
    "bind": "loopback",
    "port": 18789,
    "auth": { "mode": "token" },
    "http": { "endpoints": { "responses": { "enabled": false } } }
  },
  "tools": {
    "profile": "minimal",
    "deny": ["group:runtime", "group:fs", "browser", "canvas", "nodes", "cron"],
    "elevated": { "enabled": false }
  },
  "agents": {
    "defaults": {
      "sandbox": {
        "mode": "non-main",
        "scope": "session",
        "workspaceAccess": "rw",
        "docker": { "network": "none" }
      }
    }
  },
  "skills": { "enabled": false }
}
JSON
chmod 0640 "$OC_CFG_DIR/openclaw.json"
chown root:"$OC_USER" "$OC_CFG_DIR/openclaw.json"

```

```

# 5) Optional: firewall inbound to loopback only
if command -v ufw >/dev/null 2>&1; then
    ufw deny "$OC_PORT"/tcp || true
    ufw allow in on lo to 127.0.0.1 port "$OC_PORT" proto tcp || true
fi

# 6) Run OpenClaw's built-in hardening/audit steps (idempotent where possible)
# Note: requires openclaw CLI available in PATH for the executing user.
openclaw security audit || true
openclaw security audit --deep || true
openclaw security audit --fix || true

echo "OpenClaw baseline written to $OC_CFG_DIR. Store/seal the generated token
securely."

```

Why this aligns with OpenClaw docs: loopback binding, strict config validation expectations, deny-first tool policy, sandboxing with no network, and built-in security audit usage are directly described in OpenClaw documentation and CLI references. ⁶¹

Ansible hardening (idempotent baseline)

```

- name: Ensure openclaw system user exists
  ansible.builtin.user:
    name: openclaw
    system: true
    create_home: true
    home: /var/lib/openclaw
    shell: /usr/sbin/nologin

- name: Create config directory
  ansible.builtin.file:
    path: /etc/openclaw
    state: directory
    owner: root
    group: openclaw
    mode: "0750"

- name: Generate gateway token (one-time)
  ansible.builtin.command: openssl rand -hex 32
  register: oc_token
  changed_when: false

- name: Write secrets env
  ansible.builtin.copy:
    dest: /etc/openclaw/secrets.env
    content: "OPENCLAW_GATEWAY_TOKEN={{ oc_token.stdout }}\n"

```

```

owner: root
group: openclaw
mode: "0640"

- name: Deploy conservative openclaw.json
  ansible.builtin.copy:
    dest: /etc/openclaw/openclaw.json
    owner: root
    group: openclaw
    mode: "0640"
    content: |
      {
        "gateway": { "bind": "loopback", "auth": { "mode": "token" } },
        "tools": {
          "profile": "minimal",
          "deny": ["group:runtime", "group:fs", "browser", "canvas", "nodes",
"cron"],
          "elevated": { "enabled": false }
        },
        "agents": {
          "defaults": {
            "sandbox": { "mode": "non-main", "scope": "session", "docker":
{ "network": "none" } }
          }
        },
        "skills": { "enabled": false }
      }

- name: Run OpenClaw security audit (best-effort)
  ansible.builtin.command: openclaw security audit --deep
  register: oc_audit
  changed_when: false
  failed_when: false

- name: Print audit output
  ansible.builtin.debug:
    var: oc_audit.stdout

```

This encodes the same “hard controls” emphasized in OpenClaw’s docs: tool restriction, sandboxing, and continuous auditing. ⁶²

Terraform snippet (provider-agnostic hardening on an existing host)

This Terraform uses `remote-exec` as a “hardening applicator.” You still need separate infrastructure code to provision the host and its firewall rules; the key security requirement is that the gateway is loopback-only and not publicly reachable. ⁶³


```

terraform {
  required_providers {
    null = { source = "hashicorp/null" }
  }
}

variable "host" { type = string }
variable "ssh_user" { type = string }
variable "ssh_key_path" { type = string }

resource "null_resource" "openclaw_hardening" {
  connection {
    type      = "ssh"
    host      = var.host
    user      = var.ssh_user
    private_key = file(var.ssh_key_path)
  }

  provisioner "remote-exec" {
    inline = [
      "set -euo pipefail",
      "sudo mkdir -p /etc/openclaw",
      "sudo chmod 0750 /etc/openclaw",
      "sudo chgrp openclaw /etc/openclaw || true",
      "TOKEN=$(openssl rand -hex 32)",
      "echo OPENCLAW_GATEWAY_TOKEN=$TOKEN | sudo tee /etc/openclaw/secrets.env",
      ">/dev/null",
      "sudo chmod 0640 /etc/openclaw/secrets.env",
      "sudo tee /etc/openclaw/openclaw.json >/dev/null <<'JSON'\n{\n",
      "\"gateway\": { \"bind\": \"loopback\", \"auth\": { \"mode\": \"token\" } },\n",
      "\"tools\": { \"profile\": \"minimal\", \"deny\": [\"group:runtime\",",
      "\"group:fs\", \"browser\", \"nodes\", \"cron\"], \"elevated\": { \"enabled\":",
      "false } },\n",
      "\"agents\": { \"defaults\": { \"sandbox\": { \"mode\": \"non-",
      "main\", \"scope\": \"session\", \"docker\": { \"network\": \"none\" } } },\n",
      "\"skills\": { \"enabled\": false }\n}\nJSON",
      "sudo chmod 0640 /etc/openclaw/openclaw.json",
      "openclaw security audit --deep || true",
    ]
  }
}

```

Detection, validation, and testing recipes

Built-in validation (recommended baseline)

OpenClaw provides security and diagnostic commands that should be treated as first-line validation:

- `openclaw security audit` (config + local state) ⁶⁴
- `openclaw security audit --deep` (best-effort live gateway probe) ⁶⁴
- `openclaw security audit --fix` (tightens defaults + chmod state/config) ⁶⁴
- `openclaw secrets audit` (checks for plaintext residues, unresolved refs, precedence drift) ⁶⁴
- `openclaw doctor --deep` for deeper health checks and security warnings ⁶⁵

Control validation workflow

```
flowchart TD
    A[Change config / add channel / add skill] --> B[openclaw doctor --deep]
    B --> C[openclaw security audit]
    C --> D[openclaw security audit --deep]
    D --> E[openclaw secrets audit]
    E --> F{Findings?}
    F -- Yes --> G[Fix config + rerun audit --fix]
    F -- No --> H[Deploy + monitor]
```

This workflow is explicitly supported by OpenClaw's CLI design and the trusted-proxy migration guidance recommending audit use during deployment changes. ⁶⁶

SAST and dependency scanning

Recommended toolchain (example):

- Semgrep ⁶⁷ (SAST)
- CodeQL (source-driven analysis; also useful for taint flows)
- `npm audit` (dependency advisories)
- OSV/GHSA monitoring via the advisory stream in GitHub ⁶⁸

OpenClaw's advisory stream is active and extensive; integrate monitoring into CI and treat upgrades as part of security posture. ⁶⁹

How-to (illustrative commands)

```
# Dependency scanning
npm audit --audit-level=moderate

# Semgrep (example rulesets; tailor to Node/TS + security)
semgrep --config=auto --error
```

```
# CodeQL (high level; requires CodeQL CLI setup and language database)
# codeql database create db-openclaw --language=javascript --command="npm ci &&
npm test"
# codeql database analyze db-openclaw --format=sarif-latest --output=codeql-
results.sarif <queries>
```

What to focus on in OpenClaw-style systems (based on documented vulnerabilities and hardening notes): - WebSocket auth/origin validation paths (trusted-proxy and browser-origin logic). ⁷⁰

- URL fetch/webhook delivery paths for SSRF guard consistency. ²⁴
- Skill/plugin loading, path resolution, and secret injection paths. ¹⁴

DAST and regression tests for critical control paths

Because browser-origin and proxy interactions are recurring themes, validate with targeted tests:

- **Proxy identity path tests (trusted-proxy):** follow the trusted-proxy guide's recommendation to test headers with curl and then validate Control UI connections; re-run `openclaw security audit`. ³⁴
- **Loopback WebSocket hardening:** ensure token auth is required and rate limits are effective; recent research showed browser-origin interactions can matter even for loopback-bound services. ⁷¹
- **Browser relay exposure:** if you enable browser relay, explicitly verify the `/cdp` endpoint is protected and versions are patched, as per CVE details. ²³

Minimal "smoke" checks (examples):

```
# Confirm gateway binds only to loopback
ss -ltnp | grep 18789 || true

# Confirm security audit is clean (review output manually)
openclaw security audit --deep

# If OpenResponses HTTP endpoint enabled, confirm it is not reachable from
untrusted networks
# (run from a different host on the same network and ensure it fails)
curl -i http://<host>:18789/v1/responses
```

Fuzzing strategies (where to invest)

Given the recurring bug classes, fuzzing investment is most valuable in: - WebSocket connect/auth state machines and origin validation branches. ⁷²

- URL parsing, fetch policies, and redirect handling for SSRF resistance. ²⁴
- Skill/package parsing and sandbox boundary enforcement. ⁷³

If you stand up custom fuzzing, treat it as regression infrastructure: OpenClaw releases can be frequent, so persistent fuzz/regression tests pay off more than one-time manual reviews. ⁷⁴

Operational policies, governance, and remediation roadmap

Operational policies that preserve both safety and capability

Skill onboarding policy (recommended) - Default deny for third-party skills; require review and pin versions. ⁴⁶

- Treat marketplace scanning as a triage signal; require manual review for any skill that: - downloads external code, - touches credentials, - enables runtime tools, - changes routing/config, or - operates on webhook/URL inputs. ⁴⁸

- Require sandboxed execution for skills that touch runtime/filesystem tools. ⁷⁵

Least privilege policy - Use lowest tool profile that supports the workflow; prefer per-channel/per-sender restrictions for mixed-ingress cases. ⁷⁶

- Separate gateways for different trust boundaries instead of trying to “policy your way” into hostile multi-user safety. ²

Secrets and rotation policy - Treat any gateway token/password as operator-level. ³²

- Rotate gateway credentials and provider API keys on any suspicious event; run `openclaw secrets audit` routinely. ⁵²

Patching cadence policy - Monitor advisories continuously and upgrade quickly; the advisory stream is active and releases are frequent. ⁶⁹

- Define an “emergency patch window” measured in **hours/days** for critical/high issues affecting your enabled surfaces (browser relay, trusted-proxy, HTTP APIs). ⁷⁷

Governance controls

- **Audit:** run built-in audits on a schedule and after every config change; store outputs in an immutable log store. ⁵³
- **Change control:** treat enabling a tool (especially exec/browser/nodes/cron) or enabling a new channel ingress as a privileged change requiring review. ⁷⁸
- **Segregation:** keep personal identities/credentials off shared or experimental gateways; OpenClaw audit guidance explicitly recommends isolating shared-user setups and keeping private credentials off that runtime. ⁷⁹

Prioritized remediation roadmap (risk × effort)

Priority	Action	Risk reduced	Effort	Capability impact
Immediate	Run <code>openclaw security audit --deep</code> and fix findings; re-run <code>--fix</code>	Misconfig foot-guns; unsafe defaults; drift	Low	Usually none; may disable unsafe conveniences ⁵³
Immediate	Ensure gateway is loopback/private only; remove public exposure	Remote compromise surface	Low-Med	Lower remote convenience; restore via tunnel/VPN ⁶³

Priority	Action	Risk reduced	Effort	Capability impact
Immediate	Disable unnecessary high-risk surfaces (browser relay, HTTP APIs, open DMs)	Browser-origin, credential theft, unauthorized control	Low-Med	Reduced web automation / programmatic API / open access ⁸⁰
Near-term	Enable sandboxing for non-main sessions (no network by default)	Host blast-radius reduction	Med	Some workflows need egress rework; possible overhead ¹³
Near-term	Implement skill allowlist + review + version pinning	Supply-chain compromise	Med	Slower feature adoption; more process ⁶
Medium-term	Split gateways by trust boundary and workload class	Prevents unsafe multi-user sharing	Med-High	Operational overhead; more infra
Continuous	Automate advisory monitoring + rapid upgrades	Repeated auth/origin/policy bug class mitigation	Med	None directly; requires ops maturity ⁶⁹

Ambiguity note: This roadmap assumes you can restructure infrastructure freely (user stated no constraints). In heavily constrained environments, you may need intermediate compensating controls (e.g., heavy tool restrictions on shared gateways), but OpenClaw's own guidance still recommends splitting trust boundaries for adversarial/mixed-trust scenarios. ²

¹ ² ³ ⁸ ¹⁰ ¹⁷ ¹⁸ ²¹ ³³ ³⁶ ⁵⁴ ⁵⁸ ⁵⁹ ⁶³ **Security**

https://docs.openclaw.ai/gateway/security?utm_source=chatgpt.com

⁴ ²⁰ ²² ⁷¹ **1-Click RCE via Authentication Token Exfiltration From ...**

https://github.com/openclaw/openclaw/security/advisories/GHSA-g8p2-7wf7-98mq?utm_source=chatgpt.com

⁵ ²⁵ ⁷⁰ **CVE-2026-32302**

https://www.tenable.com/cve/CVE-2026-32302?utm_source=chatgpt.com

⁶ ¹⁴ ¹⁵ ⁴⁶ ⁷³ **Skills**

https://docs.openclaw.ai/tools/skills?utm_source=chatgpt.com

⁷ ¹³ ³¹ ⁴⁴ ⁵¹ ⁵⁶ ⁷⁵ **Sandboxing**

https://docs.openclaw.ai/gateway/sandboxing?utm_source=chatgpt.com

⁹ ²⁷ ⁶⁰ ⁶¹ **Configuration - OpenClaw**

https://docs.openclaw.ai/gateway/configuration?utm_source=chatgpt.com

¹¹ ⁶² ⁷⁶ ⁷⁸ **Tools - OpenClaw**

https://docs.openclaw.ai/tools?utm_source=chatgpt.com

¹² ²⁹ ³⁷ ⁵⁵ **Configuration Reference**

https://docs.openclaw.ai/gateway/configuration-reference?utm_source=chatgpt.com

16 **Threat Model**

https://trust.openclaw.ai/trust/threatmodel?utm_source=chatgpt.com

19 26 72 **WebSocket shared-auth connections could self-declare ...**

https://github.com/openclaw/openclaw/security/advisories/GHSA-rqpp-rjj8-7wv8?utm_source=chatgpt.com

23 45 47 80 **CVE-2026-28458 Detail - NVD**

https://nvd.nist.gov/vuln/detail/CVE-2026-28458?utm_source=chatgpt.com

24 **CVE-2026-27488 Detail - NVD**

https://nvd.nist.gov/vuln/detail/CVE-2026-27488?utm_source=chatgpt.com

28 34 57 **Trusted proxy auth**

https://docs.openclaw.ai/gateway/trusted-proxy-auth?utm_source=chatgpt.com

30 38 **Groups**

https://docs.openclaw.ai/channels/groups?utm_source=chatgpt.com

32 **OpenResponses API**

https://docs.openclaw.ai/gateway/openresponses-http-api?utm_source=chatgpt.com

35 **trustedProxy.loopbackUser for CLI/sub-agent access ...**

https://github.com/openclaw/openclaw/issues/26007?utm_source=chatgpt.com

39 50 67 79 **security**

https://docs.openclaw.ai/cli/security?utm_source=chatgpt.com

40 **IRC**

https://docs.openclaw.ai/channels/irc?utm_source=chatgpt.com

41 69 **Security Advisories**

https://github.com/openclaw/openclaw/security/advisories?utm_source=chatgpt.com

42 **Docker - OpenClaw**

https://docs.openclaw.ai/install/docker?utm_source=chatgpt.com

43 68 **FAQ - OpenClaw**

https://docs.openclaw.ai/help/faq?utm_source=chatgpt.com

48 **OpenClaw Partners with VirusTotal for Skill Security**

https://openclaw.ai/blog/virustotal-partnership?utm_source=chatgpt.com

49 **Plugins - OpenClaw**

https://docs.openclaw.ai/tools/plugin?utm_source=chatgpt.com

52 53 64 66 **CLI Reference**

https://docs.openclaw.ai/cli?utm_source=chatgpt.com

65 **doctor**

https://docs.openclaw.ai/cli/doctor?utm_source=chatgpt.com

74 **Releases · openclaw/openclaw**

https://github.com/openclaw/openclaw/releases?utm_source=chatgpt.com

77 **Browser Relay /cdp websocket missing auth could allow ...**

https://github.com/openclaw/openclaw/security/advisories/GHSA-mr32-vwc2-5j6h?utm_source=chatgpt.com